AmogaNext & AmogDS — Integration & Sync Guide

Real-World Consumer Architecture, Feature Contribution & Automated Synchronization

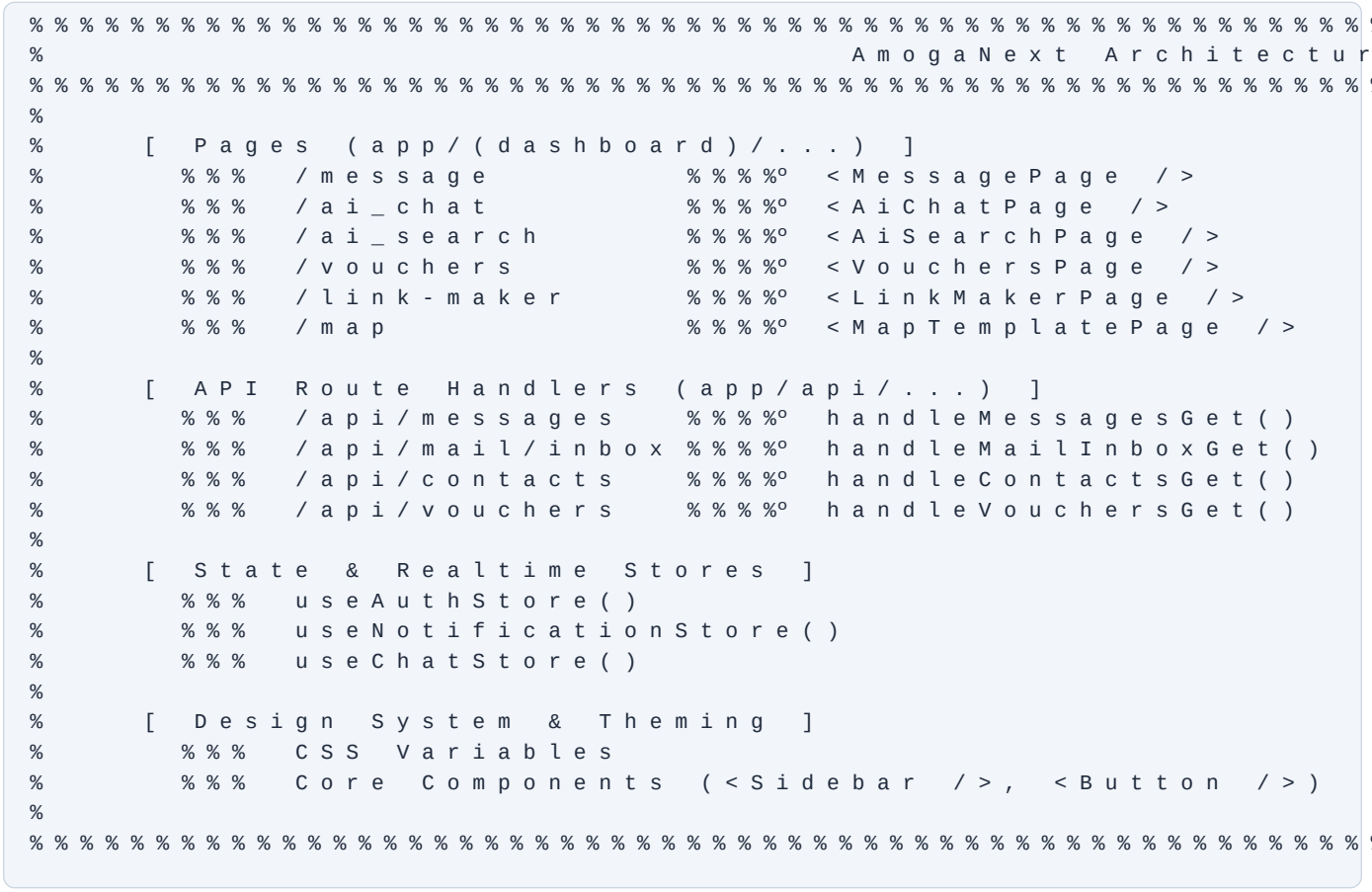
Author: Amoga Engineering Team | Version: 1.0.2 | Date: 8/25/2026

AmogaNext & AmogDS — Real-World Integration & Synchronization Workflow

Developer Architecture, Feature Contribution & Dependency Sync Guide

1. How AmogaNext Consumes AmogDS

AmogaNext is the flagship enterprise consumer of AmogDS. It has been transformed from a monolithic codebase into a thin, decoupled application that imports its UI, state, pages, and backend handlers from @amogads/ui.



2. Directory Mapping Comparison

Responsibility	AmogDS Source of Truth (`amogads/`)	AmogaNext Implementation (`shadcn-admin-main/
Core UI Components	`amogads/src/design-system/components/ui/`	Re-exported via `src/components/ui/* -> expor
Theme & CSS Tokens	`amogads/src/design-system/theme.css`	Imported in `app/globals.css` via `@amogads/u
Standard Feature Pages	`amogads/src/features/[FeatureName]/`	Wrapped in `app/(dashboard)/[feature]/page.ts
Backend API Logic	`amogads/src/server/*.handler.ts`	Dispatched in `app/api/[feature]/route.ts` vi
Realtime & State Stores	`amogads/src/stores/*.ts`	Re-used in `src/stores/*` or imported from `@

3. Step-by-Step: Adding a New Feature to AmogDS & Syncing to Consumer Apps

When you want to build a new capability (for example, a new InvoiceGenerator feature),
follow this standard 5-step workflow:

Step 1: Create the Feature in `amogads/`

Create your components and hooks in amogads/src/features/Invoice/:

```
// amogads/src/features/Invoice/index.tsx
'use client'

import React from 'react'
import { Button } from '../../design-system/components/ui/button'

export function InvoiceFeature() {
  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold">Invoices</h1>
      <Button>Create Invoice</Button>
    </div>
  )
}
```

Step 2: Register in Standard Pages

Export the page in amogads/src/standard-pages/index.ts:

```
// amogads/src/standard-pages/index.ts
export * from './invoice' // exports InvoicePage
```

Step 3: Add Backend Handler (If API Required)

Create a handler in amogads/src/server/invoice.handler.ts and export it in amogads/src/server/index.ts:

```
// amogads/src/server/invoice.handler.ts
import { NextResponse } from 'next/server'

export async function handleInvoicesGet(request: Request) {
  // Query Supabase or database
  return NextResponse.json({ success: true, invoices: [] })
}
```

Step 4: Run the Build & Sync Command

In the root directory (e:\morrai\shadcn-admin-main), run:

```
npm run sync:amogads
```

What this automated script does:

1. Compiles TypeScript bundles and `.d.ts` declaration maps using `tsup`.
2. Emits `dist/index.js`, `dist/pages.js`, `dist/server.js`, `dist/stores.js`, `dist/sdk.js`.
3. Copies all builds, type definitions, and packages into `node_modules/@amogads/ui`.

Step 5: Consume in AmogaNext (or Any Consumer)

In AmogaNext, create the page and API route:

```
// app/(dashboard)/invoices/page.tsx
'use client'

import { InvoicePage } from '@amogads/ui/pages'

export default function Page() {
  return <InvoicePage />
}
```

```
// app/api/invoices/route.ts
import { handleInvoicesGet } from '@amogads/ui/server'

export async function GET(request: Request) {
  return handleInvoicesGet(request)
}
```

4. Production Publishing & Remote Repository Synchronization

When publishing AmogDS for remote projects (outside the local monorepo workspace):

4.1 Bump Version & Publish to NPM

In amogads/:

```
cd amogads
npm version patch # e.g. 1.0.2 -> 1.0.3
npm run build:package
npm publish
```

4.2 Update in Consumer Applications

In any standalone consumer app:

```
npm update @amogads/ui
# or
npm install @amogads/ui@latest
```

5. Architectural Guardrails for Developers

1. **Safe Context Fallbacks**:

Hooks like useSidebar(), useSearch(), and useLayout() must always provide fallback objects rather than throwing unhandled exceptions when evaluated in isolation or during hydration.

2. **Supabase Realtime Unique Channels**:

When subscribing to Postgres change feeds in client components, always use uniquely generated channel topics (notifications-${userId}-${Date.now()}-${random}) to prevent cannot add postgres_changes callbacks after subscribe() exceptions.

3. **No Direct DOM Assumptions in Stores/SDK**:

Keep stores and services DOM-independent (no direct references to window or document without checks) so they can execute seamlessly in React Native and Node.js server environments.